

LISTA DOBLEMENTE ENLAZADA

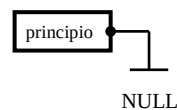
Una lista doblemente enlazada es un conjunto de datos ubicados uno a continuación del otro, con la particularidad de que un dato tiene un vínculo con el dato anterior y otro con el posterior, estos vínculos son punteros que indican la posición de memoria en la que están ubicados los datos correspondientes.

Estas listas se derivan de las simplemente enlazadas. Presentando la ventaja de que como existen dos enlaces ésta se puede reconstruir en el caso de que se halle roto algún enlace cosa que no se puede realizar en una lista simplemente enlazada. Debido al doble enlace los algoritmos usados con ellas son más complejos que en la anterior.

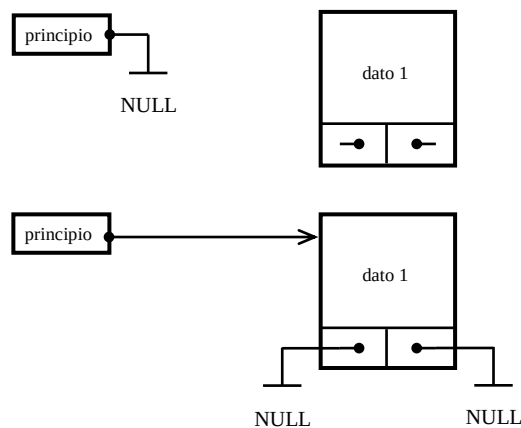
Debe ser una estructura que contenga los campos necesarios para almacenar los datos y además debe contener dos punteros a la estructura que apunten al elemento anterior y al posterior.

Al igual que en las listas simplemente enlazadas los datos se ubican en direcciones de memoria dinámica utilizando las funciones correspondientes, y la cantidad máxima de datos solo esta limitada por el modelo de memoria disponible.

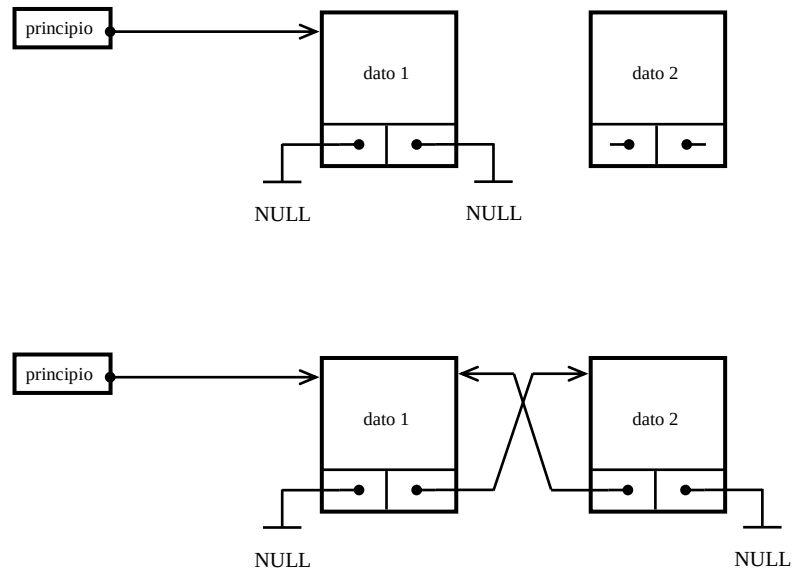
Para construir una lista doblemente enlazada se debe tener un puntero a estructura que apunte al primer conjunto de datos, este debe contener inicialmente un valor NULL indicando el estado inicial de la lista, es decir vacía.



Inicialmente el primer elemento a ubicar tendrá sus dos punteros apuntando a NULL y el indicador de comienzo apuntará a este.



El segundo elemento de la lista será incluido al final de la misma haciendo que el puntero a siguiente del nuevo elemento apuntará a NULL, el puntero a anterior del nuevo apuntará al elemento anterior y el puntero a siguiente del elemento anterior apuntará al nuevo.

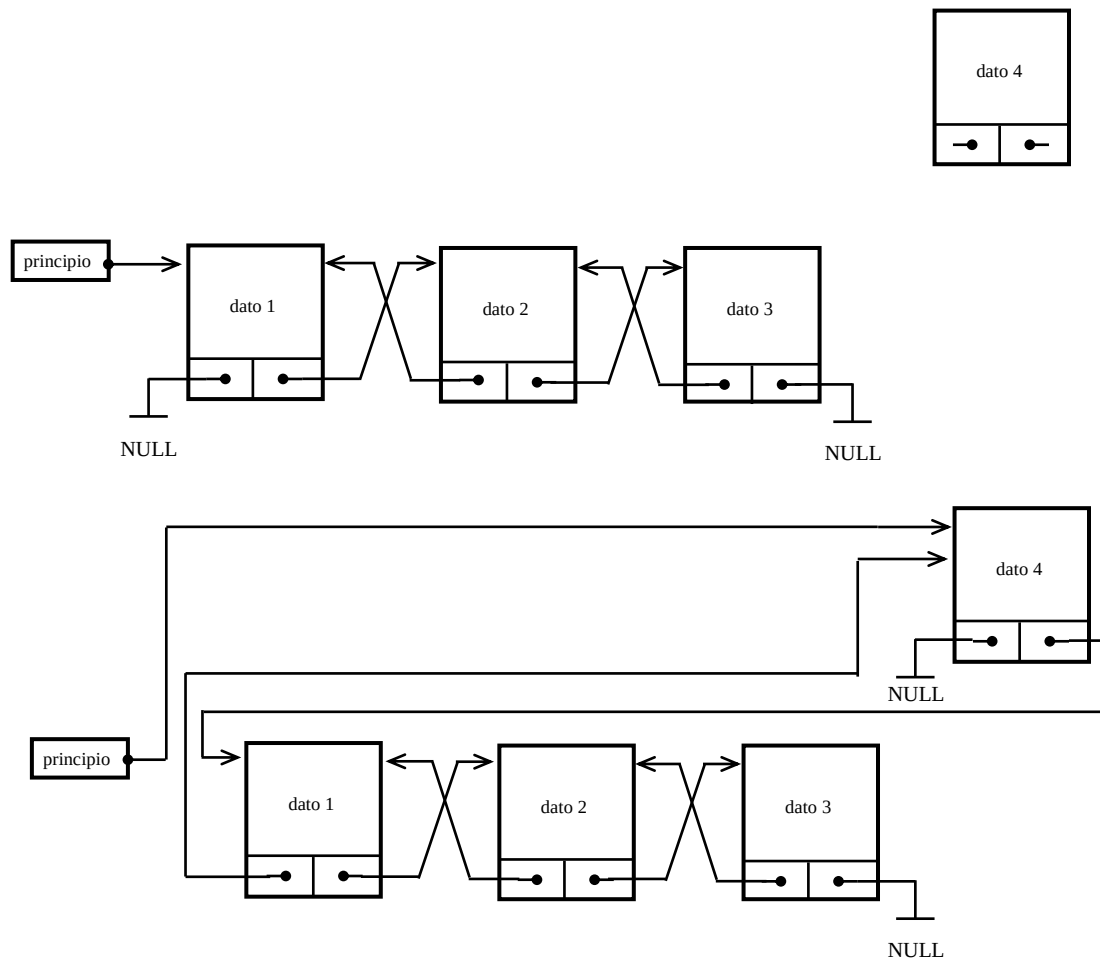


Y así sucesivamente se van agregando los nuevos elementos, repitiendo el proceso anteriormente descrito.

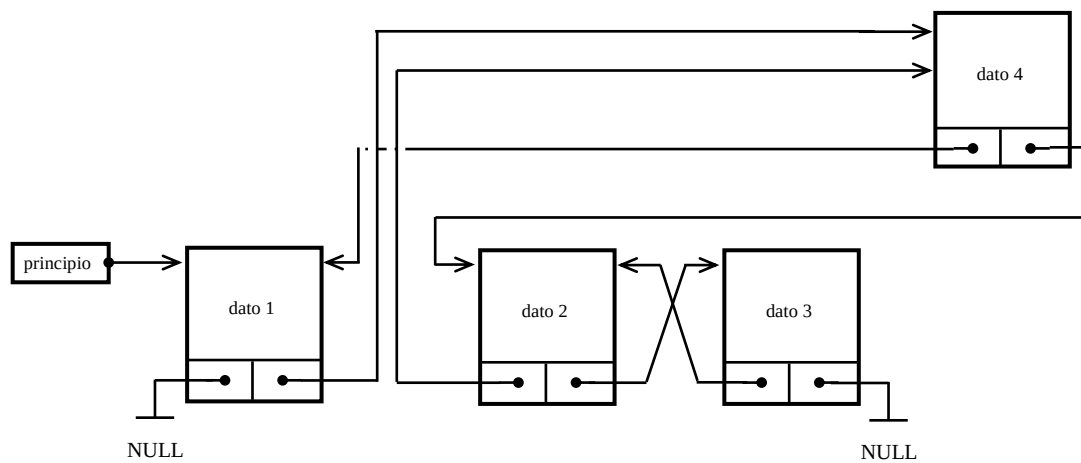
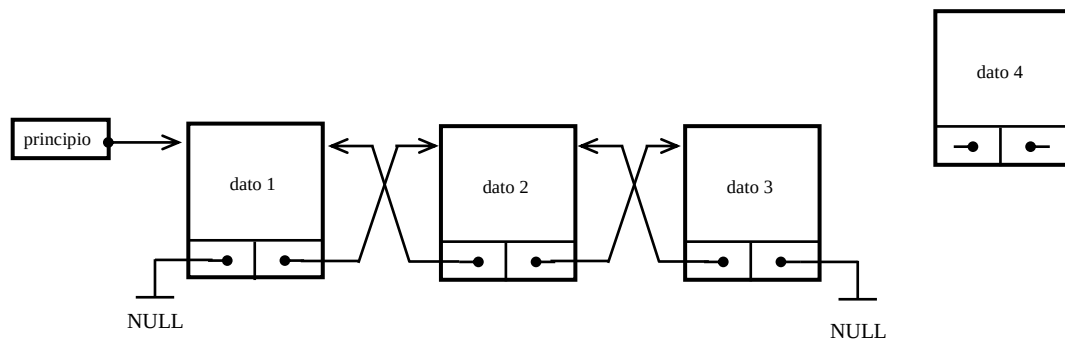
De esta forma los datos se ingresan sin ningún tipo de orden preestablecido. Para algunas aplicaciones es muy útil poder generar una lista con un orden preestablecido para ello primero debe ingresar el nuevo valor, encontrar la ubicación correspondiente para efectuar la inserción luego ubicar los punteros en el lugar adecuado. Pueden suceder tres distintos posibles casos:

- En el primer lugar.
- Entre dos elementos.
- Al final de la lista.

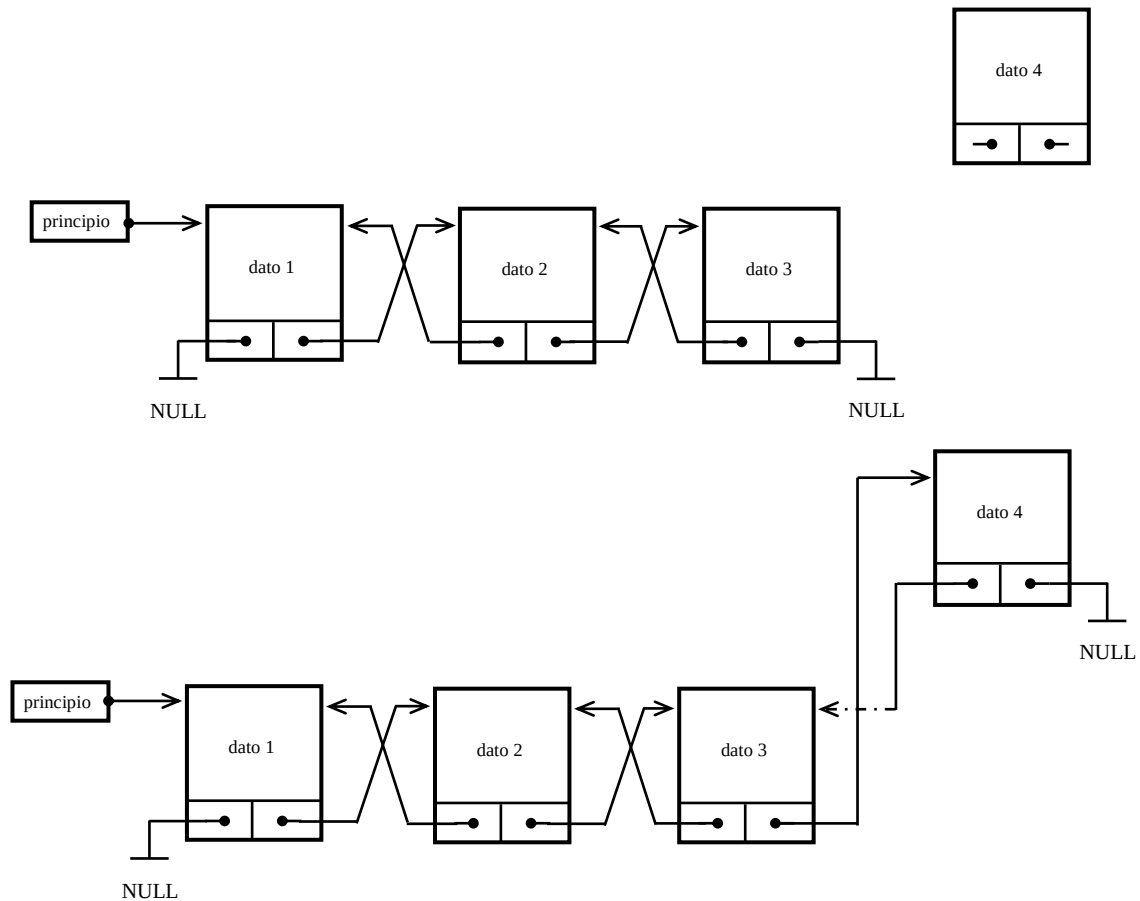
Si esta ubicado en el comienzo de la lista se debe hacer que el puntero anterior del nuevo elemento tiene que contener un NULL, el puntero siguiente debe apuntar al viejo primer elemento, y el puntero anterior del viejo primer elemento deberá apuntar al nuevo elemento.



En el caso de tener que ubicarlo entre dos elementos, cada puntero del nuevo deberá apuntar al elemento que le precede y al que lo sigue y cada puntero (anterior y siguiente), correspondiente a los elementos que pertenecían a la lista deberá apuntar al nuevo elemento respectivamente.



Cuando se debe ubicar el elemento al final de la lista, el puntero a siguiente apuntará a NULL y el puntero a anterior deberá apuntar al elemento que estaba ubicado en el final, el puntero a siguiente de este elemento deberá apuntar al nuevo elemento.



Cuando se desea imprimir la lista lo que se debe hacer es comenzar a recorrer la lista desde el comienzo hasta encontrar un puntero a siguiente con el contenido NULL.

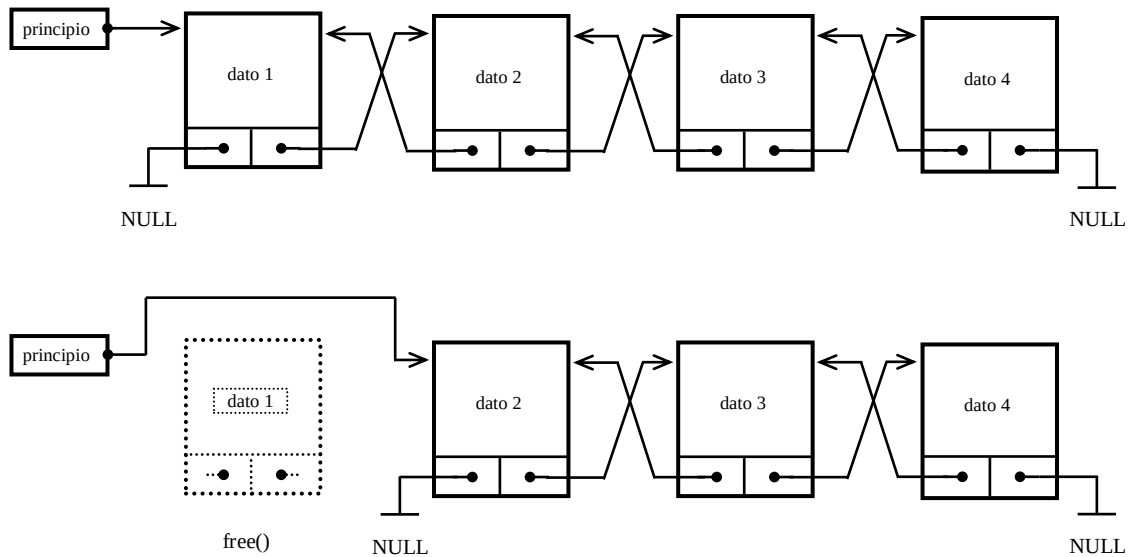
Para buscar un elemento en ella se la debe recorrer hasta encontrar el valor buscado. En el caso en que el valor deseado no se encuentre en ella se debe informar tal situación.

En el caso de tener que borrar un elemento hay que actualizar el puntero anterior y siguiente con los valores que van a quedar y luego proceder a liberar la memoria.

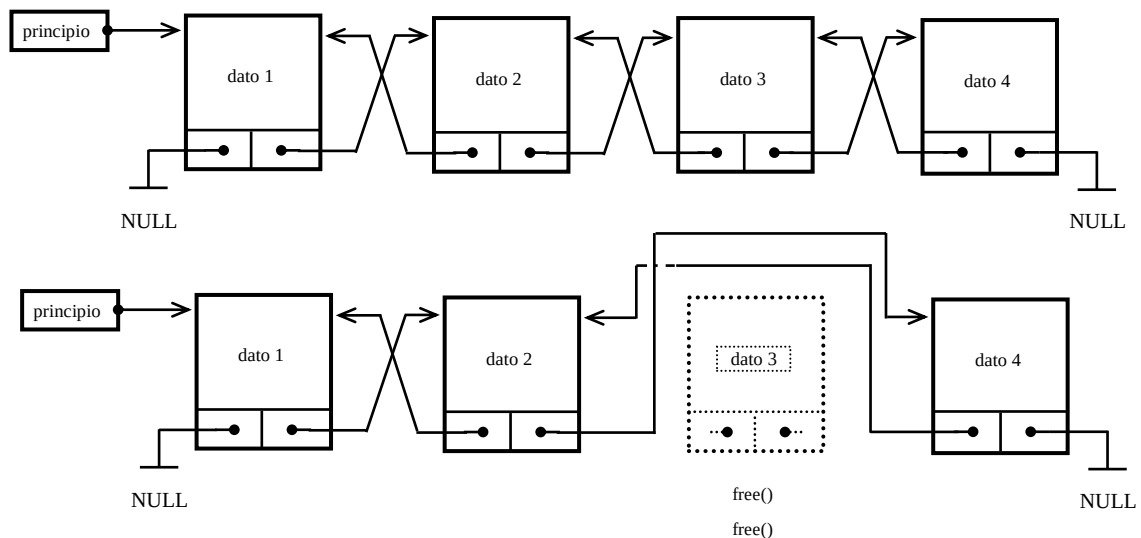
En esta situación se pueden producir tres variantes:

- Borrar el primer elemento.
- Borrar un elemento intermedio.
- Borrar el último elemento.

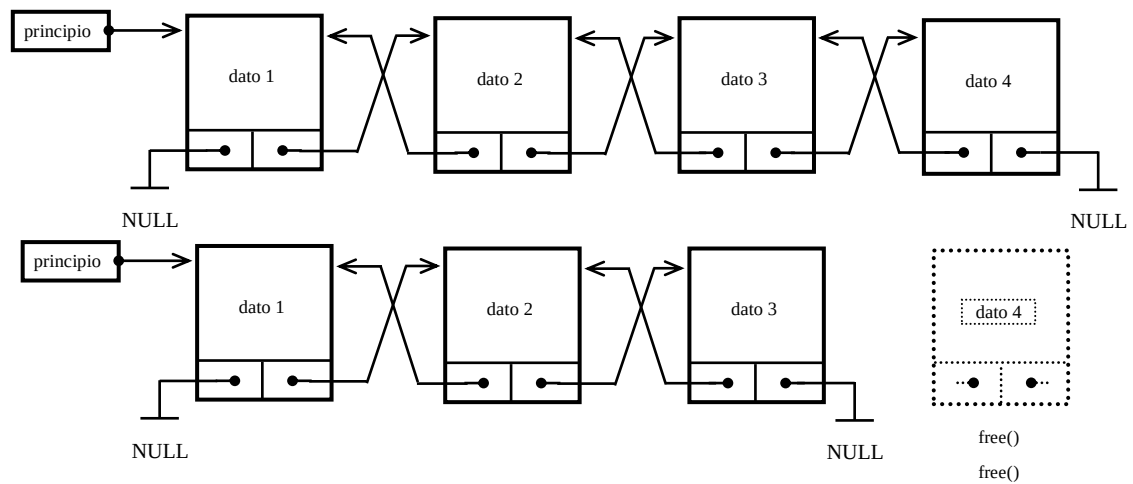
Para borrar el primer elemento hay que proceder a modificar el puntero a inicio con el valor de la posición del segundo elemento, a continuación hay que colocar un valor NULL en el puntero anterior del segundo elemento y por ultimo hay que liberar la memoria que ocupa el elemento a borrar.



Cuando se desee eliminar un elemento intermedio hay que actualizar el puntero siguiente del elemento que precede al que hay que eliminar con el contenido del siguiente anterior del elemento a borrar, el puntero anterior del elemento que sucede al que hay que borrar se lo debe actualizar con el valor del puntero anterior de este y luego hay que liberar la memoria ocupada por elemento eliminado.



En el caso de que sea el último elemento de la lista hay que colocar un NULL en el puntero a siguiente del elemento anterior al borrado y luego proceder a liberar la memoria de la ubicación del elemento a borrar.



Ejemplo

```

1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <conio.h>
4.  struct lid { int nrosocio;
5.               char categoria;
6.               char nombre[20];
7.               char apellido[20];
8.               char direccion[40];
9.               char telefono[14];
10.             struct lid *sig;
11.             struct lid *ant;};
12. void ingresa(struct lid **);
13. void lee(struct lid **);
14. void borra(struct lid **);
15. void busca(struct lid **);
16. void endat(struct lid **);
17. void impdat(struct lid *);
18. void main(void)
19. {
20.     int op;
21.     struct lid *pr=NULL;
22.     for(;;)
23.     {
24.         clrscr();
25.         printf("1 - Ingresar datos\n");
26.         printf("2 - Leer datos\n");
27.         printf("3 - Borrar datos\n");
28.         printf("4 - Buscar datos\n");
29.         printf("5 - Salir\n\n");
30.         printf("Ingrese una opcion ( 1 - 5 ) : ");
31.         fflush(stdin);
32.         scanf("%d",&op);
33.         switch(op)
34.         {
35.             case 1: ingresa(&pr);
36.                     break;
37.             case 2: lee(&pr);
38.                     break;

```

```

39.         case3: borra(&pr);
40.             break;
41.         case4: busca(&pr);
42.             break;
43.         case5: exit(0);
44.     }
45. }
46. }
47. void ingresa(struct lid **p)
48. {
49.     struct lid *act,*aux,*ant;
50.     char k;
51.     clrscr();
52.     do
53.     {
54.         aux=(struct lid *)malloc(sizeof(struct lid));
55.         if(aux)
56.         {
57.             printf("\n\nIngrese los datos : \n\n");
58.             endat(&aux);
59.             if(!*p)
60.             {
61.                 *p=aux;
62.                 aux->sig=NULL;
63.                 aux->ant=NULL;
64.             }
65.             else
66.             {
67.                 ant=*p;
68.                 act=*p;
69.                 while((act->nrosocio<aux->nrosocio)&&act)
70.                 {
71.                     ant=act;
72.                     act=act->sig;
73.                 }
74.                 if(!act)
75.                 {
76.                     ant->sig=aux;
77.                     aux->sig=NULL;
78.                     aux->ant=ant;
79.                 }
80.                 else if(ant==act)
81.                 {
82.                     *p=aux;
83.                     aux->sig=ant;
84.                     aux->ant=NULL;
85.                     ant->ant=aux;
86.                 }
87.                 else
88.                 {
89.                     ant->sig=aux;
90.                     aux->sig=act;
91.                     aux->ant=ant;
92.                     act->ant=aux;
93.                 }
94.             }
95.         }
96.     }
97.     {

```

```

98.         clrscr();
99.         printf("\n\nNo hay memoria disponible\n");
100.        printf("\n\nPresione una tecla para continuar\n");
101.        getch();
102.        return;
103.    }
104.    printf("\nOtro dato ( S / N )? ");
105.    k=getche();
106.    k=toupper(k);
107.    clrscr();
108.    }
109.    while(k=='S');
110. }
111. void lee(struct lid **p)
112. {
113.     struct lid *aux=*p;
114.     while(aux)
115.     {
116.         clrscr();
117.         printf("\nLos datos son : \n\n");
118.         impdat(aux);
119.         aux=aux->sig;
120.         printf("\n\nPresione una tecla para continuar\n");
121.         getch();
122.     }
123. }
124. void borra(struct lid **p)
125. {
126.     struct lid *aux,*ant;
127.     unsigned b;
128.     char k;
129.     do
130.     {
131.         clrscr();
132.         printf("\nIngresa el nro de socio a borrar : ");
133.         scanf("%d",&b);
134.         aux=*p;
135.         ant=*p;
136.         while((b!=aux->nrosocio)&&aux)
137.         {
138.             ant=aux;
139.             aux=aux->sig;
140.         }
141.         if(aux)
142.         {
143.             clrscr();
144.             printf("\nLos datos son : \n\n");
145.             impdat(aux);
146.             printf("\n\nDesea eliminar estos datos (S / N) ");
147.             k=getche();
148.             k=toupper(k);
149.             if(k=='S')
150.             {
151.                 if(ant==aux)
152.                 {
153.                     *p=aux->sig;
154.                     aux->sig->ant=NULL;
155.                 }
156.                 else

```

```

157.         {
158.             ant->sig=aux->sig;
159.             ant->sig->ant=ant;
160.         }
161.         free(aux);
162.     }
163. }
164. else
165. {
166.     clrscr();
167.     printf("\nLos datos no han sido encontrados");
168.     printf("\n\nPresione una tecla para continuar\n");
169.     getch();
170. }
171. printf("\nOtro dato ( S / N )? ");
172. k=getche();
173. k=toupper(k);
174. }
175. while(k=='S');
176. }
177. void busca(struct lid **p)
178. {
179.     struct lid *aux;
180.     unsigned b;
181.     char k;
182.     do
183.     {
184.         clrscr();
185.         printf("\nIngresa el nro de socio a buscar : ");
186.         scanf("%d",&b);
187.         aux=*p;
188.         while((b!=aux->nrosocio)&&aux)
189.             aux=aux->sig;
190.         if(aux)
191.         {
192.             clrscr();
193.             printf("\nLos datos son : \n\n");
194.             impdat(aux);
195.             printf("\n\nPresione una tecla para continuar\n");
196.             getch();
197.         }
198.         else
199.         {
200.             clrscr();
201.             printf("\nLos datos no han sido encontrados");
202.             printf("\n\nPresione una tecla para continuar\n");
203.             getch();
204.         }
205.         printf("\nOtro dato ( S / N )? ");
206.         k=getche();
207.         k=toupper(k);
208.     }
209.     while(k=='S');
210. }
211. void endat(struct lid **dato)
212. {
213.     char n[255],c;
214.     printf("\nNro. de socio : ");
215.     scanf("%d",&(*dato)->nrosocio);

```

```

216. do
217. {
218.     clreol();
219.     printf("\nCategoria      : ");
220.     c=getche();
221.     c=toupper(c);
222. }
223. while(c!='M'&&c!='C'&&c!='I');
224. (*dato)->categoria=c;
225. fflush(stdin);
226. do
227. {
228.     clreol();
229.     printf("\n\nNombre      : ");
230.     gets(n);
231. }
232. while(strlen(n)>=20);
233. strcpy((*dato)->nombre,n);
234. fflush(stdin);
235. do
236. {
237.     clreol();
238.     printf("\nApellido      : ");
239.     gets(n);
240. }
241. while(strlen(n)>=20);
242. strcpy((*dato)->apellido,n);
243. fflush(stdin);
244. do
245. {
246.     clreol();
247.     printf("\nDirección      : ");
248.     gets(n);
249. }
250. while(strlen(n)>=40);
251. strcpy((*dato)->direccion,n);
252. do
253. {
254.     clreol();
255.     printf("\nTeléfono      : ");
256.     gets(n);
257. }
258. while(strlen(n)>=14);
259. strcpy((*dato)->telefono,n);
260. }
261. void impdat(struct lid *dato)
262. {
263.     printf("\nNro. de socio : %d",dato->nrosocio);
264.     printf("\nCategoria      : %c",dato->categoria);
265.     printf("\nNombre      : %s",dato->nombre);
266.     printf("\nApellido      : %s",dato->apellido);
267.     printf("\nDirección      : %s",dato->direccion);
268.     printf("\nTeléfono      : %s",dato->telefono);
269. }

```

En el programa expuesto se puede visualizar la utilización de una lista doblemente enlazada para generar un conjunto de datos de personas.

Estos datos se ubican en una estructura, en esta también se definen dos punteros a este tipo de estructura, uno llamado ant y otro denominado sig, que apuntaran a los elementos anterior y siguiente de la lista. La estructura esta definida entre las líneas 4 a 11 y se denomina lid.

La función main esta definida entre las líneas 18 a 46, esta permite las opciones siguientes:

- Ingresar datos
- Leer datos
- Borrar datos
- Buscar datos
- Salir

A estas opciones se puede ingresar a través del uso de un menú y de una estructura del tipo switch-case con el cual se invoca a las funciones correspondientes. Cuando se elige la opción de salir se invoca a la función exit con un parámetro 0 que indica una terminación normal del programa.

La función ingresa permite introducir los datos en la posición adecuada según el orden preestablecido con el campo nrosocio, se define entre las líneas 47 a 110, su prototipo se ubica en la línea 12. Recibe como parámetro un puntero a puntero a struct lid denominado p, en este se guarda el valor de la dirección del puntero al comienzo de la lista; devuelve un valor void.

Tiene un lazo para permitir el ingreso de mas de un juego de datos cada vez que se la invoque mediante la elección del operador.

Como primer paso se solicita memoria para la estructura y el resultado de este pedido se guarda en el puntero aux. Si este contiene un valor NULL indica que el pedido no fue satisfactorio por lo tanto se indica tal situación y se regresa a la función main (líneas 97 a 103).

Si el pedido de memoria tuvo éxito se evalúa tal situación en la línea 55 y luego se introducirán los datos invocando a la función endat enviándole la dirección del bloque solicitado (líneas 57 y 58).

A continuación se evalúa el contenido de p para determinar si es el primer elemento de la lista (línea 59), asignándole a este el valor de aux y a los punteros aux->sig y aux->ant el valor NULL (líneas 61 a 63).

Cuando la lista contiene por lo menos algún elemento se inicializan a los punteros ant y act con el contenido de p (líneas 67 y 68), para comenzar la búsqueda del lugar en donde insertar el nuevo elemento esto se hace usando un ciclo while con una condición (act->nrosocio<aux->nrosocio)&&act, de la que se sale cuando se encuentre el lugar correspondiente o el fin de la lista (líneas 69 a 73). Al encontrarse el lugar correspondiente el puntero act lo contendrá y su valor puede ser NULL, que indica el final de la lista, o distinto de NULL que indicaría la posición del elemento siguiente al nuevo.

En la línea 74 se determina la condición de fin de lista, de cumplirse se actualizan los punteros ant->sig con el valor de aux, aux->sig con NULL y aux->ant con el valor ant, creando los enlaces correspondientes al ultimo elemento (líneas 76 a 78).

Si los punteros ant y act son iguales esto indicara que es el primer elemento, en cuyo caso se asigna al contenido de p el valor del nuevo elemento (aux), al puntero aux->sig el valor de ant (anterior primer elemento), a aux->ant el valor NULL y a ant->ant se actualiza con aux (líneas 82 a 85).

De ser un elemento intermedio se actualizan los punteros ant->sig y act->ant con aux, es decir ambos apuntan al nuevo elemento, al puntero aux->sig con act y a aux->ant con ant (líneas 89 a 92).

La función lee se define entre las líneas 111 a 123 y permite imprimir en pantalla los datos de toda la lista a esta se le envía como parámetro la dirección del puntero que indica el comienzo de la lista y devuelve void.

Al puntero aux se le asigna el valor del contenido de p y luego se ingresa a un lazo while y se mantendrá en él siempre que el valor del puntero sea distinto a NULL.

Dentro del lazo se imprimen en pantalla los datos usando la función impdat y se actualiza al puntero aux con el valor aux->sig, para continuar el recorrido de la lista.

El prototipo de lee se encuentra en la línea 13.

La función borra esta definida entre las líneas 124 y 176, su prototipo esta en la línea 14, ella recibe un puntero a puntero a struct lid denominado p y en él se tiene la dirección del puntero a inicio de la lista. Devuelve el valor void.

Tiene un lazo do-while para permitir borrar mas de un elemento a petición del operador. Se debe ingresar el valor de un campo del elemento a borrar (líneas 131 a 133). Luego se inicializan los valores de los punteros ant y aux con el contenido de p (inicio de la lista) esto se realiza en las líneas 134 y 135. A continuación se ingresa a un lazo while que tendrá la condición (b!=aux->nrosocio)&&aux la cual mantiene en funcionamiento al lazo hasta que se encuentre el elemento buscado o se encuentre el final de la lista. Dentro del cuerpo del while se modifican los valores de los punteros ant con el valor de aux y al puntero aux con el valor aux->sig para poder recorrerla (líneas 136 a 140).

Si aux tiene un valor distinto a NULL, indica que se encontró este elemento, esto se evalúa en la línea 141.

De ocurrir esto se imprimen los datos mediante la función impdat y se pregunta si se desea eliminar (líneas 143 a 148). De ser afirmativo se procede a eliminar este elemento.

Para ello se comprueba si los punteros aux y ant son iguales, lo que indicaría que hay que eliminar el primer elemento de la lista, esto se hace modificando el contenido de p con el puntero aux->sig y luego el puntero aux->sig->ant con NULL (esto significa lo siguiente: aux->sig es lo mismo que el elemento siguiente y por lo tanto aux->sig->ant es lo mismo que ant->ant) esto se hace entre las líneas 151 a 155.

En caso contrario el elemento esta ubicado en una posición intermedia o en el final de la lista, en ambos casos se modifican los punteros ant->sig con aux-sig y ant->sig->ant con ant (líneas 158 y 159).

Para todos los casos se debe liberar la memoria mediante el uso de la función free (línea 161).

Desde la línea 166 a 169 se realiza la operación que indica que el elemento no ha sido encontrado, esta situación se avisa y luego se retornará a main.

Entre las líneas 177 y 210 se define la función busca que permite encontrar un elemento en la lista. Esta es similar a la función borra pero no elimina ningún elemento de la lista, solo lo presenta en la pantalla. Su prototipo esta en la línea 15.

La función endat se define entre las líneas 211 y 260, recibe la dirección del puntero en donde se van a guardar los datos y devuelve void. El prototipo esta ubicado en la línea 16.

Esta función realiza la tarea de ingreso de los datos con una validación de los mismos. Desde la línea 261 a la 269 se define la función impdat que permite la impresión en pantalla de los datos recibiendo la dirección de la estructura y devolviendo void. El prototipo se encuentra en la línea 17.

```
1. void ingresa(struct lid **p)
2. {
3.     struct lid *act,*aux,*ant;
4.     char k;
5.     clrscr();
6.     do
7.     {
8.         aux=(struct lid *)malloc(sizeof(struct lid));
9.         if(aux)
10.        {
11.            printf("\n\nIngrese los datos : \n\n");
12.            endat(&aux);
13.            if(!*p)
14.            {
15.                *p=aux;
16.                aux->sig=NULL;
17.                aux->ant=NULL;
18.            }
19.            else
20.            {
21.                act=*p;
22.                ant=*p;
23.                while(act)
24.                {
25.                    ant=act;
26.                    act=act->sig;
27.                }
28.                ant->sig=aux;
29.                aux->sig=NULL;
30.                aux->ant=ant;
31.            }
32.        }
33.        else
34.        {
35.            clrscr();
36.            printf("\n\nNo hay memoria disponible\n");
37.            printf("\n\nPresione una tecla para continuar\n");
38.            getch();
39.            return;
40.        }
41.        printf("\nOtro dato ( S / N )? ");
42.        k=getche();
43.        k=toupper(k);
44.        clrscr();
45.    }
46.    while(k!='S');
47. }
```

la función `ingresa` escrita arriba se utiliza para agregar un elemento al final de la lista. Difiere de la anterior en que no evalúa la condición de primer elemento ni la de elemento intermedio. Los parámetros que recibe son los mismos que en la función descrita en el programa.